

Task 4: RESTful API with Spring Boot - Comprehensive Explanation

Project Context

You're building the backend engine for "ProductivityPro Suite" - a task management system. This replaces the Servlet/JSP contact manager with a modern, scalable API that will later power web/mobile apps.

Core Responsibilities Breakdown

1. Spring Boot Setup & Configuration

- Objective: Create production-ready foundation
- Actions:
 - Initialize project with essential dependencies (Web, Data JPA, Security)
 - Configure database connection in `application.properties`
 - Set up project structure (entities, repositories, controllers)
- Outcome:

```
taskflow-api/  
├─ src/main/java/com/productivitypro/  
│   ├── controller/    # REST endpoints  
│   ├── model/         # JPA entities  
│   ├── repository/    # Spring Data interfaces  
│   └─ exception/      # Custom error handling  
└─ src/main/resources/
```

- └─ `application.properties`

2. REST Controller Development

- Objective: Expose CRUD operations via HTTP
- Actions:
 - Implement endpoints for:
 - `GET /tasks` → List all tasks
 - `POST /tasks` → Create new task

- PUT /tasks/{id} → Update task
 - DELETE /tasks/{id} → Remove task
 - Use appropriate HTTP status codes (200, 201, 404)
- Code Example:

```
@RestController
@RequestMapping("/api/tasks")
public class TaskController {

    @GetMapping
    public List<Task> getAllTasks() {
        return taskRepository.findAll();
    }

    • }
```

3. Spring Data JPA Implementation

- Objective: Simplify database operations
- Actions:
 - Create Task entity with JPA annotations
 - Define repository interface:
 - public interface TaskRepository extends JpaRepository<Task, Long> {}
 - Leverage built-in methods: .save(), .findById(), .delete()
- Key Benefit: Eliminates 90% of JDBC boilerplate code

4. HATEOAS Implementation

- Objective: Make API self-discoverable
- Actions:
 - Add hypermedia links to responses

```
EntityModel<Task> model = EntityModel.of(task);
```

- model.add(linkTo(methodOn(TaskController.class).getTaskById(id)).withSelfRel());
 - Enable clients to navigate API dynamically
- Response Example:

```
{
```

```

    "id": 1,
    "title": "Learn HATEOAS",
    "_links": {
        "self": {"href": "https://api.example.com/tasks/1"},
        "tasks": {"href": "https://api.example.com/tasks"}
    }
}

```

- }

5. Global Exception Handling

- Objective: Unified error management
- Actions:
 - Create `@ControllerAdvice` class
 - Handle exceptions:
 - `TaskNotFoundException` → 404 Not Found
 - `MethodArgumentNotValidException` → 400 Bad Request
 - Return structured error responses:

```

{
    "status": 404,
    "message": "Task not found with id: 99"
}

```

- }

6. Spring Security Integration

- Objective: Protect API endpoints
- Actions:
 - Configure HTTP Basic authentication
 - Restrict access to authenticated users
 - Password encryption with BCrypt
 - Set up in-memory users for development:

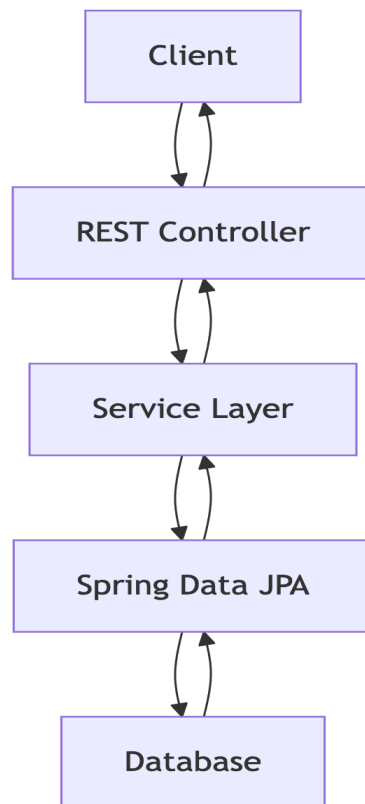
```

UserDetails user = User.withUsername("admin")
                        .password(encoder.encode("secret"))
                        .roles("USER")

```

- `.build();`

Key Technical Concepts



Dependency Injection in Action

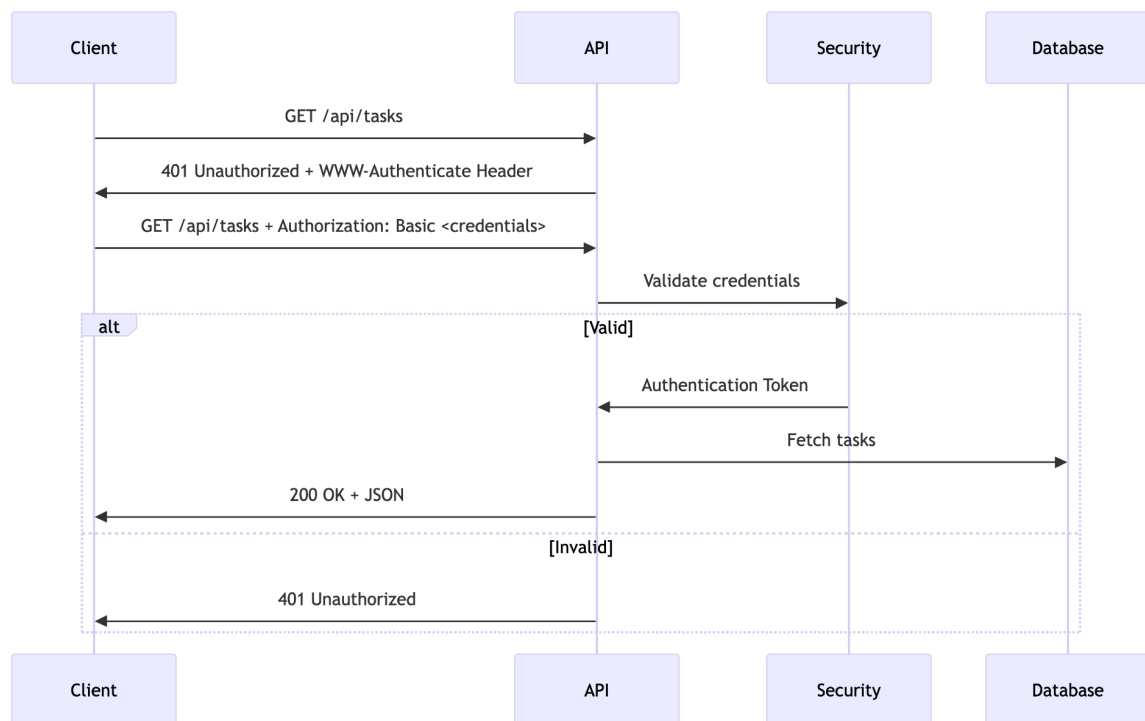
- How it works:
 1. Spring creates `TaskRepository` bean
 2. Autowires it into `TaskController`
 3. Controller uses repository without manual instantiation
- Code:

```
@RestController
public class TaskController {
    @Autowired // Injected by Spring
    private TaskRepository repository;
```

- }
-

Security Implementation Details

Authentication Flow



Password Security

- Never store plaintext passwords
- BCrypt hashing process:
 1. Generate random salt
 2. Hash password + salt
 3. Store format:
\$2a\$10\$N9qo8uL0ickgx2ZMRZoMyeIjZAgcf17p92ldGxad68LJZdL17lhWy
- |-- Algorithm --|-- Cost --|---- Salt ----|----- Hash -----|

Validation Mechanism

Declarative Validation

```
public class Task {
```

```
@NotBlank(message="Title required")
@Size(max=100)
private String title;

@FutureOrPresent
private LocalDate dueDate;
}
```

Error Response Structure

```
{
  "errors": [
    {
      "field": "dueDate",
      "message": "Due date must be in future"
    },
    {
      "field": "title",
      "message": "Title required"
    }
  ]
}
```

Testing & Quality Assurance

1. Endpoint Testing (Postman/Insomnia):
 - Verify all CRUD operations
 - Test authentication/authorization
 - Check validation responses
2. Automated Tests:
 - Controller unit tests with `@WebMvcTest`
 - Security tests with `@WithMockUser`
 - Repository tests with `@DataJpaTest`
3. API Documentation:
 - Auto-generated Swagger UI at `http://localhost:8080/swagger-ui.html`

Why This Matters

1. Modern Backend Standards:
 - RESTful design principles
 - Stateless authentication
 - Hypermedia-driven navigation
 2. Production Readiness:
 - Built-in metrics with Actuator
 - Externalized configuration
 - Health checks
 3. Foundation for Scaling:
 - Easy containerization (Docker)
 - Cloud deployment readiness
 - Microservice-friendly architecture
-

Success Metrics

Checkpoint	Verification Method
CRUD operations functional	Postman test suite
100% authenticated endpoints	Unauthenticated access attempts
Validation error handling	Invalid payload submissions
HATEOAS links in responses	Inspect JSON output
Password security	Verify password hashing in database
API documentation available	Access /swagger-ui.html

Final Output: A secure, self-documenting REST API ready for frontend integration in Task 5.

